Deploying the AIMScribe Server to DigitalOcean

The complete manual: accounts, droplet, storage, the stack, the endpoints, the clinic PCs, and keeping it running · AIMS LAB · 15 October 2026

This manual moves the **whole AIMScribe v3 server** — nine services, both databases, the gateway, the archive worker, the backups and the monitor — onto one DigitalOcean droplet in Bangalore, with the audio in Cloudflare R2. It is written for someone who has never run a Linux server: every step says **what to type**, **what you should see**, and **what to do if you see something else**.

	Stage	Time	Who
0	**Accounts** — DigitalOcean and Cloudflare, owned by one email, with two-factor login	30 min	the owner
A	**The endpoint status table** — what works before and after. Give this to CMED	read it now	you → CMED
B	Before you start: the checklist, the two R2 buckets, the plan	45 min	AIMS LAB
C	**The deployment, twelve steps**	**3–4 hours**	AIMS LAB
D	Proving every endpoint answers	20 min	AIMS LAB
E	**Daily and weekly checks** — one page	5 min a day	AIMS LAB
F	Updates, rollback, what was fixed, go-live checklist	30 min	AIMS LAB

The one thing to understand before starting

CMED's page never learns the server address. It talks to the recorder on the same PC, over loopback. Only **CMED's own server** and **each recorder** hold the AIMS LAB address. So this deployment changes exactly two things on the outside: one setting in CMED's server, and the server address in each recorder (set by re-running the installer, step 11). CMED's page is not touched.

How to read the boxes in this manual

Grey box, typewriter text — type it exactly. Lines starting with # are explanations; you may type them or leave them out.
<angle brackets> — replace with your own value, without the brackets.
Green text — what a correct result looks like.
"If you see this instead" tables — the common failures, and the fix for each.
Commands that start with PS> are typed in **Windows PowerShell on your own laptop**. Everything else is typed **on the droplet**, over SSH.

PART 0

Accounts

Two accounts, one owner email, two-factor login on both

0.1 · The owner email

Every account and every automatic notice in this deployment goes to one address, so nothing important lands in a mailbox nobody reads:

Where the email is used	Why
DigitalOcean account owner	billing, droplet alerts, password resets
Cloudflare account owner	R2 storage, billing, API token notices
`AIMS_ACME_EMAIL` **in the server settings (step 6)**	Let's Encrypt writes here if the HTTPS certificate is about to expire
DigitalOcean monitoring and uptime alerts (step 12)	CPU, memory, disk, and "the server stopped answering"
The comment on your SSH key (step 1)	so you can tell whose key it is later

```
Owner email:   your-team-email@example.org
```

The server's own alerts do not go to email. The monitor service sends its alerts (disk filling, recordings not confirmed, backups missing) to a **webhook** — `AIMS_ALERT_WEBHOOK` — such as a Slack, Discord or Google Chat incoming-webhook address. There is no email setting in the stack. Email alerts come from DigitalOcean's own monitoring, set up in step 12.

0.2 · DigitalOcean

	Do this	Where
1	Sign up with **your-team-email@example.org**. Add a payment card	cloud.digitalocean.com → Sign up
2	**Turn on two-factor login** (an authenticator app, not SMS)	avatar → My Account → Security → Two-factor authentication
3	Create a team named **AIMS LAB**; invite each lab member who will administer the server, as **Member** (not Owner)	Settings → Team → Invite members
4	Set a **billing alert** at $150 a month, so a mistake shows before the invoice does	Billing → Billing alerts
5	Create a project named **AIMScribe**; everything in this manual goes in it	left menu → New Project

If you see this instead	What it means, and what to do
"Account under review" after sign-up	DigitalOcean checks new accounts. It can take a few hours. Nothing else in this manual can start until it clears.
The card is declined	Some Bangladeshi cards need international online payments enabled by the bank first. Or use PayPal in the same billing page.

0.3 · Cloudflare

	Do this	Where
1	Sign up with **your-team-email@example.org**	dash.cloudflare.com → Sign up
2	**Turn on two-factor login**	My Profile → Authentication
3	Open **R2 Object Storage** and add a payment method. R2 has a free tier, but a card is required to enable it	left menu → R2 Object Storage
4	Note your **Account ID**. It is part of the storage address	R2 Object Storage overview page, right-hand side
5	Invite lab members who need it, with the narrowest role that works	Manage Account → Members

Write the recovery codes down
Both services give **recovery codes** when two-factor login is turned on. Store them with the access sheet, off this laptop. Losing the phone without them can lock AIMS LAB out of its own server and its own recordings for days.

PART A

The endpoint status table

What works today, what works after deployment - the page to send CMED

1 · Every endpoint, before and after

This is the table to put in front of CMED. "Today" means the system as it runs now, on a PC on a private network. "After" means once Part C is done.

	Endpoint	Today	After	Why
1	`GET https://<host>/health`	**NO**	**YES**	Needs a public name and a certificate. Both arrive with the droplet. No key
2	`GET https://<host>/`	**NO**	**YES**	same
3	`POST https://<host>/api/v2/clinical/patient-information` *API 2*	**NO**	**YES**	**The one CMED is waiting for.** Finished and tested; it has had no public address to answer on. Header `X-CMED-Key`
4	`POST https://<host>/api/v2/clinical/prescription` *API 3, Channel B*	**NO**	**YES**	same as 3
5	`ws://127.0.0.1:5050/ws` — `start` *API 1*	**YES**	**YES**	**Unaffected.** Loopback on the clinic PC — never involved our server
6	`ws://127.0.0.1:5050/ws` — `prescription_built` *API 3, Channel A*	**YES**	**YES**	Unaffected
7	`ws://127.0.0.1:5050/ws` — `doctors`	**YES**	**YES**	Unaffected
8	`ws://127.0.0.1:5050/ws` — `status`	**YES**	**YES**	Unaffected
9	`GET https://<host>/api/v2/clinical/patient/{id}/visits` *new: what we hold*	**NO**	**YES**	same as 3. No key: **401** `INVALID_KEY`
10	`POST https://<host>/api/v2/session/reassign` *new: recorders only*	—	**YES**	Not CMED's. Needs schema script 013. No device token: **401**
11	`POST https://<host>/api/v2/device/health` *new: recorders only*	—	**YES**	Not CMED's. Needs schema script 014. No device token: **401**

Read the shape of that table. The four local ones already work and are untouched by the deployment. The five server ones CMED calls are blocked by one cause — no public address — and are unblocked by one act. Rows 10 and 11 are the recorders' own, and need nothing from CMED.

2 · What API 2 and API 3 Channel B answer

The replies are the same before and after deployment. These are the ones CMED's outbox must act on, including what happens when it sends the same visit twice:

CMED sends	Reply	What it means / what CMED does
A new visit (API 2) or prescription (API 3b)	**202** `ACCEPTED`	Stored. Remove it from the outbox. For API 2, `previous_visit.status` says whether the history was `ALREADY_STORED`, `UPDATED` or `STORED`

CMED sends	Reply	What it means / what CMED does
The identical body again (a retry after a timeout, or after an outage)	**200** `ALREADY_RECEIVED`	**Nothing new stored.** It was already here. Remove it from the outbox
API 2 for the same visit (same five fields) with a different body	**202** `ACCEPTED`	Stored as a newer record of that visit; **the newest is the one used for matching**. Remove it from the outbox
A prescription for the same visit with a changed body	**202** `ACCEPTED`, `"version": 2`	Kept as version 2 — a correction. Earlier versions are kept too
The same prescription rebuilt — a new `issued_at`, keys reordered	**200** `ALREADY_RECEIVED`	Same fingerprint, so nothing new. Remove it from the outbox
No key, or a wrong one	**401**	Do not retry. Check the key
Body over 1 MB	**413**	Do not retry. Report it to AIMS LAB
A field malformed, or a previous visit named by only some of its five fields	**422** `SCHEMA_INVALID`	Kept aside for review. Do not retry the same body; fix and send again
Our server down, the branch's internet down, or a network error	no reply / **5xx**	**Keep it in the outbox and retry later.** Late is fine: it attaches to its recording whenever it lands

The five fields are `patient_id`, `doctor_id`, `hospital_id`, `start_time` and `date`. The full rules are in CMED_1, the software integration document.

3 · What to say to CMED, in writing

A note you can send as it stands:

```
The four endpoints your page uses on the clinic PC work today and are not
affected by our deployment. Your front-end team can build and test against
them now - install the recorder on one PC and they are live:

    ws://127.0.0.1:5050/ws   commands: start, prescription_built,
                                       doctors, status

The endpoints your server uses are finished and tested, and will answer
as soon as our DigitalOcean deployment is done:

    POST  https://<host>/api/v2/clinical/patient-information   (API 2)
    POST  https://<host>/api/v2/clinical/prescription          (API 3, Channel B)
    GET   https://<host>/api/v2/clinical/patient/{id}/visits - what we hold
    GET   https://<host>/health              - no key, to check we are up

While a branch has no internet, keep both POSTs in an outbox on its
local server and send them when it returns. Late is fine.

Nothing about the message format, the reply codes, the key or the limits
changes between now and then. Please hold the host as configuration, not
a compiled-in constant: we will give you a staging host first and a
production host later.
```

PART B

Before you start

The checklist, the storage buckets, and what will be running

4 · What you need in hand

	What	Where from	Note
■	The two accounts from Part 0, with two-factor login on	Part 0	
■	**A domain name you control**	a UIU subdomain from UIU IT, or a registrar	**Required.** A certificate cannot be issued for a bare IP address
■	The two R2 buckets and their tokens	section 6 below	four values, each shown once
■	An SSH key pair	your laptop, step 1	password login will be turned off
■	CMED's code for each clinic	CMED	**needed at step 10**
■	The list of clinic laptops (clinic, room)	the clinics	**needed at step 11**
■	A password manager or the access sheet	AIMS LAB	**six secrets must be stored off the server** (step 6)

The domain is the only hard prerequisite. The gateway asks Let's Encrypt for a certificate the first time it starts, and that only works when the name already points at the droplet. This manual uses `aimscribe.uiu.ac.bd` as the example name; use whatever name you actually get.

5 · What will be running when you are done

Service	What it does	Exposed?
gateway (Caddy)	HTTPS, the certificate, security headers, a 2 MB body cap	**yes — 80 and 443**
api	every endpoint, on internal port 6000	no
postgres	both databases: `aims_recordings` and `aims_clinical`	no
pgbouncer	connection pooling	no
redis	the work queue	no
worker	transcription (needs the Azure settings; optional)	no
archive-worker	pulls the pieces of each recording, joins them, keeps the archive copy and uploads the encrypted cloud copy	no
backup	nightly dump of both databases, encrypted — **does not run in its image yet; step 12 replaces it**	no
monitor	disk, unconfirmed recordings, certificate, backups	no

Only the gateway is reachable from outside, on ports 80 and 443. Every other service talks on a private Docker network that has no route to the internet except through the gateway.

6 · Cloudflare R2: two buckets, two tokens

The stack needs two separate buckets, not one. The server refuses to make cloud copies if both settings name the same bucket (it logs a critical error and makes none).

Bucket	What goes in it	Setting it fills
`aimscribe-segments`	The short pieces each recorder uploads while it records. The recorders upload straight to it, using a short-lived signed address from our API	`MINIO_BUCKET`
`aimscribe-archive`	One encrypted copy of each finished consultation — the permanent cloud archive (the 3 TB)	`AIMS_COPY_BUCKET`

	Do this in Cloudflare	Result
1	R2 Object Storage → **Create bucket** → name `aimscribe-segments` → Location: **Automatic**, with the hint **Asia-Pacific** if offered → Create	bucket 1
2	Same again for `aimscribe-archive`	bucket 2
3	Leave **Public access off** on both. Never enable a public r2.dev address	private
4	R2 → **Manage R2 API Tokens** → Create API token → name `aimscribe-segments-rw` → permission **Object Read & Write** → **Apply to specific buckets only**: `aimscribe-segments` → Create	**Access Key ID** and **Secret Access Key** — shown once
5	Same again, named `aimscribe-archive-rw`, for `aimscribe-archive` only	a second pair — shown once
6	Copy the **S3 endpoint** shown with the token. It looks like `https://<account-id>.r2.cloudflarestorage.com`	you will use it **without** the `https://`

If you see this instead	What it means, and what to do
The token page shows only "Account API tokens" and "User API tokens"	Either works. Pick **Object Read & Write** and restrict it to the one bucket.
The secret is gone after closing the page	It is shown once. Delete that token and create a new one.
A "Bucket lock" or retention option is offered on `aimscribe-archive`	Worth turning on for the archive bucket only: it stops anyone deleting the permanent copies. Never on the segments bucket, which the server empties as it works.

Keep the two pairs apart. The segments token can delete; the archive token must never be given to anything except this server's settings.

7 · What the 16 GB droplet needs that the default does not

The stack's defaults were written for a 64 GB machine at UIU. On the 16 GB droplet four settings must change, or the database will not stay up. The profile they point at ships with the code; step 6e sets them:

Setting	Default	On the droplet	Why
`AIMS_PG_CONF`	`postgresql.conf`	`postgresql.16gb.conf`	**The default reserves 16 GB for the database cache alone** — the whole machine. The 16 GB profile is in `deploy/uiu/postgres`, tested under a 6 GB limit
`AIMS_PG_MEMORY`	8g	**6g**	leaves room for the API and the workers
`AIMS_ARCHIVE_MEMORY`	1g	**2g**	joining a long consultation in memory, without being killed
`AIMS_WORK_TMPFS`	6g	**3g**	working space in memory for joining one consultation

PART C

The deployment

Twelve steps, three to four hours, done once

Step 1 · Make your SSH key, then create the droplet

An SSH key is how you log in to the server without a password. You make it once, on your own laptop. Windows 10 and 11 already include the tool.

1a · On your laptop, in Windows PowerShell

```
PS> ssh-keygen -t ed25519 -C "your-team-email@example.org"
#   press Enter to accept the file name
#   type a passphrase (twice) - it protects the key if the laptop is stolen

PS> Get-Content $env:USERPROFILE\.ssh\id_ed25519.pub | Set-Clipboard
#   the PUBLIC key is now on the clipboard
```

You should see: two files in `C:\Users\<you>\.ssh\`: `id_ed25519` (private — never share it, never upload it) and `id_ed25519.pub` (public — this is what DigitalOcean gets). The public key is one line starting `ssh-ed25519 AAAA`.

If you see this instead	What it means, and what to do
'ssh-keygen' is not recognized	Settings → System → Optional features → Add a feature → **OpenSSH Client**. Then open a new PowerShell window.
"id_ed25519 already exists. Overwrite?"	Answer **n**. You already have a key; use the existing `.pub` file.

1b · In DigitalOcean: Create → Droplets

Field	Choose
Region	**Bangalore · BLR1** — the lowest latency to Dhaka
Image	**Ubuntu 24.04 (LTS) x64**
Droplet type	**Basic** → **Regular (SSD)**
Size	**8 vCPU / 16 GB / 320 GB SSD —** `s-8vcpu-16gb` **— about $96 a month**
Authentication	**SSH Key** → **New SSH Key** → paste the clipboard → name it after yourself. **Not a password**
Backups	**Enable — weekly** (about 20% extra, ~$19 a month)
Monitoring	**Enable** — free; needed for the email alerts in step 12
Hostname	`aimscribe-prod-blr1`
Project	AIMScribe

You should see: after about a minute the droplet appears with a green dot and a **public IPv4 address**. Write the address down; it is `<droplet-ip>` below.

Optional: a reserved IP. Networking → Reserved IPs → assign one to the droplet, and use that address from now on. It is free while attached, and if the droplet is ever rebuilt the address — and so the DNS record — stays the same.

Step 2 · Point the name at the droplet

At whoever controls the domain (UIU IT for a `uiu.ac.bd` name), create one record:

```
Type  Name                  Value              TTL
A     aimscribe.uiu.ac.bd   <droplet-ip>       300
```

Then wait, and check from your laptop:

```
PS> nslookup aimscribe.uiu.ac.bd 1.1.1.1
```

You should see: `Address: <droplet-ip>` in the answer. Usually within 5 minutes; occasionally up to an hour.

If you see this instead	What it means, and what to do
`Non-existent domain` **or no address**	The record is not live yet, or was typed under the wrong name. Wait 10 minutes and try again; then check the record itself.
A different address	An old record exists. Delete it so only the new A record remains.

Do not start the stack (step 8) until this answers correctly. The gateway asks for a certificate on first start; if the name does not point here yet the request fails and Let's Encrypt makes you wait before trying again.

Step 3 · The firewall

DigitalOcean → Networking → **Firewalls** → Create Firewall, name `aimscribe-prod`, with exactly these rules, then apply it to the droplet:

Direction	Protocol	Port	Source / destination	Why
Inbound	TCP	**22**	**your own IP only**	SSH, administration
Inbound	TCP	**80**	All IPv4, All IPv6	certificate issue and renewal
Inbound	TCP	**443**	All IPv4, All IPv6	**the API**
Inbound	UDP	**443**	All IPv4, All IPv6	HTTP/3
Outbound	all	all	all	R2 uploads, updates, certificates

Nothing else. No database port, no Redis port, no API port. Use this DigitalOcean firewall rather than `ufw` on the droplet: Docker opens its ports around `ufw`, so `ufw` alone would give a false sense of safety.

If you see this instead	What it means, and what to do
You do not know your own IP	Search "what is my IP" from the laptop you will administer from, and use that address with `/32`.
Your IP changes (home broadband)	Add each address you use, or use the DigitalOcean web console (droplet → Access) when locked out.

Step 4 · First login, a working user, and Docker

From your laptop, log in as `root` the first time only:

```
PS> ssh root@<droplet-ip>
#   first time: "Are you sure you want to continue connecting?" -> yes
#   then your key passphrase
```

You should see: `root@aimscribe-prod-blr1:~#` — you are on the server.

Now, on the droplet, create the user you will use from now on, and lock the door behind it:

```
# a working user with sudo, and your key
adduser --disabled-password --gecos "" aims
usermod -aG sudo aims
rsync --archive --chown=aims:aims ~/.ssh /home/aims
passwd aims            # set a password - sudo will ask for it

# no root login, no password logins
sed -i "s/^#*PermitRootLogin.*/PermitRootLogin no/" /etc/ssh/sshd_config
sed -i "s/^#*PasswordAuthentication.*/PasswordAuthentication no/" /etc/ssh/sshd_config
grep -rl "PasswordAuthentication yes" /etc/ssh/sshd_config.d/ \
  | xargs -r sed -i "s/PasswordAuthentication yes/PasswordAuthentication no/"
sshd -t && systemctl restart ssh
sshd -T | grep -E "^(permitrootlogin|passwordauthentication)"
```

You should see: `permitrootlogin no` and `passwordauthentication no`.

Keep this root window open until the next login works
Open a **second** PowerShell window and run `ssh aims@<droplet-ip>`. Only when that succeeds, close the root window. If it fails, the root window is still open to fix it — and the DigitalOcean web console (droplet → Access → Launch Droplet Console) always works as a last resort.

As `aims`, the updates, the clock, a swap file and Docker:

```
ssh aims@<droplet-ip>

sudo apt update && sudo apt upgrade -y
sudo apt install -y unattended-upgrades fail2ban git curl jq python3
sudo dpkg-reconfigure -plow unattended-upgrades      # answer Yes

# the clock - recordings are filed by the clinic's local date
sudo timedatectl set-timezone Asia/Dhaka
sudo timedatectl set-ntp true

# 4 GB of swap: a cushion, so a spike never kills the database
sudo fallocate -l 4G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo "/swapfile none swap sw 0 0" | sudo tee -a /etc/fstab

# Docker, with the compose plugin
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker aims
sudo systemctl enable --now docker
exit                                  # log out and back in, so the group applies
ssh aims@<droplet-ip>
docker compose version && timedatectl | grep "Time zone" && free -h
```

You should see: `Docker Compose version v2...`, `Time zone: Asia/Dhaka`, and `Swap: 4.0Gi`.

If you see this instead	What it means, and what to do
`permission denied ... docker.sock`	You did not log out and back in after `usermod`. Run `exit`, then log in again.
`Could not get lock /var/lib/dpkg/lock`	Ubuntu is installing its own updates in the background on first boot. Wait five minutes and run the command again.
A purple screen asks which services to restart	Press Enter to accept the default.

The timezone line is not cosmetic. A recording is filed under the clinic's local date. A server on UTC files the evening's consultations under tomorrow.

Step 5 · Get the code

```
cd ~
git clone https://github.com/SakibZaman021/AIMScribe_v3.git
cd AIMScribe_v3/deploy/uiu
ls
```

You should see: `Caddyfile backup docker-compose.yml monitor pgbouncer postgres ...` and, with `ls -a`, `.env.example`.

The folder is named `uiu` for historical reasons. It is the **server** stack and the right one for this droplet. `deploy/local` is the development stack; never use it here.

Step 6 · The settings file

All settings live in one file, `deploy/uiu/.env`. It holds every secret on the machine, is readable only by you, and is **never committed** — the repository is public. Work through 6a to 6g in one SSH session, in the `deploy/uiu` folder.

6a · Start from the example, and generate the secrets on the server

```
cd ~/AIMScribe_v3/deploy/uiu
cp .env.example .env && chmod 600 .env

# setv NAME VALUE - replaces the line NAME=... in .env, or adds it
setv() { sed -i "s|^$1=.*|$1=$2|" .env; grep -q "^$1=" .env || echo "$1=$2" >> .env; }

# passwords and service keys: random, 64 hex characters each
for v in POSTGRES_SUPERUSER_PASSWORD AIMS_RECORDINGS_PASSWORD \
         AIMS_CLINICAL_PASSWORD AIMS_MONITOR_PASSWORD REDIS_PASSWORD \
         AIMS_ADMIN_KEY AIMS_WORKER_KEY AIMSCRIBE_WEBHOOK_SECRET; do
  setv $v "$(openssl rand -hex 32)"
done

# the two encryption keys: 32 random bytes each, base64
setv AIMS_BACKUP_KEY "$(openssl rand -base64 32)"
setv AIMS_COPY_KEY   "$(openssl rand -base64 32)"
```

Keep this SSH window open until 6g: `setv` exists only in this session. If the window closes, paste the `setv()` line again before continuing. Hex is used for the passwords because one of them is placed inside a database address, where `/` or `+` would break it.

6b · Who this server is

```
setv AIMS_PUBLIC_HOST aimscribe.uiu.ac.bd             # the name from step 2
setv AIMS_ACME_EMAIL  your-team-email@example.org
setv TZ               Asia/Dhaka
setv AIMS_DB_PATH      /srv/aimscribe/db
setv AIMS_ARCHIVE_PATH /srv/aimscribe/archive
setv AIMS_BACKUP_PATH  /srv/aimscribe/backup

# the exact address CMED's dashboard is served from (no wildcards);
# ask CMED - a placeholder is harmless, it only governs browser access
setv AIMS_ALLOWED_ORIGINS https://<cmed-dashboard-host>
```

6c · The two R2 buckets from Part B, section 6

```
# the endpoint is the HOST NAME ONLY - no https:// in front
R2=<account-id>.r2.cloudflarestorage.com

# the pieces, uploaded by the recorders while they record
setv MINIO_ENDPOINT   $R2
setv MINIO_BUCKET     aimscribe-segments
setv MINIO_SECURE     true
setv MINIO_REGION     auto
setv MINIO_ACCESS_KEY <segments token Access Key ID>
setv MINIO_SECRET_KEY <segments token Secret Access Key>

# the permanent encrypted copy of each consultation
setv AIMS_COPY_ENDPOINT   $R2
setv AIMS_COPY_BUCKET     aimscribe-archive
setv AIMS_COPY_REGION     auto
setv AIMS_COPY_SECURE     true
setv AIMS_COPY_ACCESS_KEY <archive token Access Key ID>
setv AIMS_COPY_SECRET_KEY <archive token Secret Access Key>
```

`.env.example` **suggests Amazon for the copy store (**`s3.ap-south-1.amazonaws.com`**).** The lines above replace that with R2, which is the policy for this deployment.

6d · The server's two signing keys

The server signs two things with private keys: each **recording grant**, and each **purge receipt** that lets a clinic PC delete audio it has safely uploaded. Every recorder checks those signatures with the matching **public** keys, which the installer copies to `C:\ProgramData\AIMScribe\keys`. **The private and public halves must be a pair**, or no recording can be authorised and no audio is ever deleted from a PC.

Make a new pair on the droplet, put the private halves in `.env`, and keep the public halves for the installer:

```
mkdir -p ~/keys && cd ~/keys && umask 077
openssl genpkey -algorithm ed25519 -out grant.pem
openssl genpkey -algorithm ed25519 -out receipt.pem
openssl pkey -in grant.pem   -pubout -out aimslab_grant_pub.pem
openssl pkey -in receipt.pem -pubout -out aimslab_receipt_pub.pem

cd ~/AIMScribe_v3/deploy/uiu
# .env holds a key on one line, with its line breaks written as \n
for k in GRANT RECEIPT; do
  f=~/keys/$(echo $k | tr A-Z a-z).pem
  sed -i "/^AIMS_${k}_PRIVATE_KEY=/d" .env
  printf "%s\n" "AIMS_${k}_PRIVATE_KEY=$(awk 'BEGIN{ORS="\\n"}1' $f)" >> .env
done

# prove each private key in .env matches its public file
for k in grant receipt; do
  K=$(echo $k | tr a-z A-Z)
  grep "^AIMS_${K}_PRIVATE_KEY=" .env | cut -d= -f2- | xargs -0 printf "%b" \
    | openssl pkey -pubout | diff -q - ~/keys/aimslab_${k}_pub.pem >/dev/null \
    && echo "$k: MATCH" || echo "$k: DIFFERENT"
done
```

You should see: `grant: MATCH` and `receipt: MATCH`.

Then copy the two **public** files to your laptop, into the recorder source, so the installer you build in step 11 carries them:

```
PS> cd <your AIMScribe_v3 folder>\recorder\keys
PS> scp aims@<droplet-ip>:keys/aimslab_grant_pub.pem .
PS> scp aims@<droplet-ip>:keys/aimslab_receipt_pub.pem .
```

Store `grant.pem` and `receipt.pem` in the password manager (6g), then delete them from the droplet with `shred -u ~/keys/grant.pem ~/keys/receipt.pem`. The copies in `.env` are the working ones.

Why new keys, and not the ones already in the repository
On 15 October 2026 the public keys in `recorder/keys` did **not** match the private keys of the AIMS LAB development server. Nobody can say for certain which server holds their private halves. Starting with a fresh pair removes the doubt, and costs nothing: every clinic PC is re-run through the installer in step 11 anyway, which installs the new public keys.

6e · Sizing for the 16 GB droplet

The database profile for this droplet is part of the code: `deploy/uiu/postgres/postgresql.16gb.conf`. Its header shows how the 16 GB is shared between the services. Point the settings at it:

```
cd ~/AIMScribe_v3/deploy/uiu
grep -E "^(shared_buffers|effective_cache_size|work_mem)" postgres/postgresql.16gb.conf

setv AIMS_PG_CONF        ./postgres/postgresql.16gb.conf
setv AIMS_PG_MEMORY      6g
setv AIMS_ARCHIVE_MEMORY 2g
setv AIMS_WORK_TMPFS     3g

# how long an archived WAV stays on this disk once its cloud copy is proven
setv AIMS_ARCHIVE_LOCAL_RETENTION_DAYS 2       # two clinic days: about 66 GB
```

You should see: `shared_buffers = 4GB`, `effective_cache_size = 10GB`, `work_mem = 16MB`.

The disk floors need nothing. Left empty, each of the monitor's floors is capped at 15% of its disk — 48 GB here — so a floor written for the UIU array no longer alarms for ever on a 320 GB disk.

6f · Alerts, and the optional parts

```
# where the monitor posts its alerts (a Slack/Discord/Google Chat webhook URL).
# Left empty, alerts go only to the log.
setv AIMS_ALERT_WEBHOOK https://<your chat webhook address>

# transcription: only if this deployment transcribes. Otherwise leave empty.
#   AZURE_TRANSCRIBE_ENDPOINT, AZURE_TRANSCRIBE_API_KEY,
#   AZURE_NER_ENDPOINT, AZURE_NER_API_KEY
```

6g · Check, then put six secrets somewhere safe

```
for v in POSTGRES_SUPERUSER_PASSWORD AIMS_RECORDINGS_PASSWORD AIMS_CLINICAL_PASSWORD \
  AIMS_MONITOR_PASSWORD REDIS_PASSWORD AIMS_WORKER_KEY AIMS_ADMIN_KEY AIMS_PUBLIC_HOST \
  AIMS_ACME_EMAIL AIMS_DB_PATH AIMS_ARCHIVE_PATH AIMS_BACKUP_PATH AIMS_BACKUP_KEY \
  AIMS_COPY_KEY AIMS_GRANT_PRIVATE_KEY AIMS_RECEIPT_PRIVATE_KEY MINIO_ENDPOINT \
  MINIO_ACCESS_KEY MINIO_SECRET_KEY AIMS_COPY_BUCKET AIMS_COPY_ACCESS_KEY \
  AIMS_COPY_SECRET_KEY AIMS_PG_CONF; do
  grep -q "^$v=." .env && echo "ok       $v" || echo "MISSING  $v"
done
grep -c "^MINIO_BUCKET=aimscribe-segments$" .env     # must print 1
docker compose config --quiet && echo "compose settings: OK"
```

You should see: every line `ok`, then `1`, then `compose settings: OK`. No secret is printed by these checks.

If you see this instead	What it means, and what to do
`MISSING <name>`	Set it with `setv` as above, then run the check again.
`required variable ... is missing a value`	One of the eleven the stack refuses to start without is empty. The message names it.
`setv: command not found`	The session was closed. Paste the `setv()` line from 6a again.

Six secrets that must also exist OFF this machine	If lost
`AIMS_COPY_KEY`	**Every cloud copy in R2 is permanently unreadable**
`AIMS_BACKUP_KEY`	**Every database backup is unreadable**
`grant.pem` / `AIMS_GRANT_PRIVATE_KEY`	every clinic PC must be re-installed with new public keys
`receipt.pem` / `AIMS_RECEIPT_PRIVATE_KEY`	same
`AIMS_ADMIN_KEY`	no administration until `.env` is read on the server
`POSTGRES_SUPERUSER_PASSWORD`	restores and repairs become much harder

To read a value to copy it into the password manager, without leaving it in the shell history: `grep ^AIMS_COPY_KEY= .env`. Do this on your own screen, never in a screen share.

Step 7 · The folders

```
sudo mkdir -p /srv/aimscribe/{db,archive,backup}
# the archive worker runs as user 10001 inside its container, and must
# be able to write the archive; the database and backup containers
# set their own ownership
sudo chown -R 10001:10001 /srv/aimscribe/archive
# the nightly backup in step 12 runs as you
sudo chown aims:aims /srv/aimscribe/backup
df -h /srv
```

You should see: about `310G` available on `/`.

Do not `chown` **the archive folder to** `aims`**.** The archive worker would then fail to write any recording, with `Permission denied` in its log.

Step 8 · Start it

```
cd ~/AIMScribe_v3/deploy/uiu
docker compose up -d --build
#   the first time builds four images: 5-10 minutes

docker compose ps
```

You should see: nine services. `aims-postgres`, `aims-pgbouncer`, `aims-redis` and `aims-api` say `(healthy)`; the rest say `Up`. The first start creates both databases and applies the whole schema by itself.

```
docker compose logs --tail 60 api
docker compose logs -f gateway        # Ctrl+C to stop watching
```

You should see: in the gateway log, `certificate obtained successfully` for your host, within a minute or two.

If you see this instead	What it means, and what to do
`aims-postgres` **restarting, or its log says** `could not map anonymous shared memory`	The 16 GB profile is not in use. Check `AIMS_PG_CONF` (6e), then `docker compose up -d`.
`aims-postgres` **log:** `could not open configuration file`	The file named in `AIMS_PG_CONF` does not exist. Check the spelling in 6e, and that `git pull` brought `postgres/postgresql.16gb.conf`.
Gateway log: `NXDOMAIN`, `timeout` **or** `rateLimited`	DNS is not pointing here yet (step 2), or port 80 is closed (step 3). Fix it, then `docker compose restart gateway`. After a rate limit, wait an hour.
API log: `AIMS_COPY_BUCKET is the same bucket as the pieces`	Both settings name one bucket. Set them as in 6c, then `docker compose up -d`.
API log: `AIMS_RECEIPT_PRIVATE_KEY is not set`	6d was not completed.
Archive-worker log: `Permission denied` **on** `/archive`	Step 7's `chown` was skipped or used the wrong user.
A service is `Exited`	`docker compose logs --tail 80 <service>` shows why. Fix the setting, then `docker compose up -d`.

Step 9 · Prove it answers — from your laptop, not the server

```
PS> curl.exe -s https://aimscribe.uiu.ac.bd/health
```

You should see: `"status":"healthy"`, `"database":"connected"`, `"redis":"connected"`, `"minio":"connected"`, `"copy_bucket":"connected"`, and `"system"` starting with `AIMScribe v3`. The two storage answers are real: the server asks each R2 bucket, with its own keys, at most every 30 seconds.

If you see this instead	What it means, and what to do
Could not resolve host	Step 2 is not finished.
A certificate or TLS error	The gateway has not got its certificate yet. Watch `docker compose logs -f gateway`.
`"status":"degraded"`	Read which part is not `connected`. The database or Redis: `docker compose ps`. `"minio"` or `"copy_bucket"` saying `refused`: that bucket's keys are wrong (6c); `bucket missing`: the bucket name is wrong; `unreachable`: the endpoint is wrong.
Connection refused or timed out	Port 443 is closed in the firewall (step 3), or the gateway is not running.

Run it from somewhere else, not from the droplet. Testing from the server proves the program is running; it does not prove that DNS, the firewall and the certificate work — which is the point of this step.

Step 10 · Register the clinics, the doctors and CMED's key

These are admin API calls, authenticated with `X-Admin-Key`. Run them on the droplet; the key is read from `.env` into a variable and never typed.

```
cd ~/AIMScribe_v3/deploy/uiu
A=$(grep ^AIMS_ADMIN_KEY= .env | cut -d= -f2)
H=https://aimscribe.uiu.ac.bd/api/v2

# one clinic. cmed_hospital_id is CMED's code for it - the mapping that
# lets CMED's visits be matched to this clinic's recordings
curl -s -X POST $H/admin/hospital -H "X-Admin-Key: $A" \
  -H "Content-Type: application/json" \
  -d '{ "hospital_id": "AALO_DHOLPUR",
```

```
        "name": "Aalo Clinic, Dholpur",
        "timezone": "Asia/Dhaka",
        "cmed_hospital_id": "<CMED code for this clinic>" }'

# repeat for AALO_KARAIL, AALO_MIRPUR, AALO_SHYAMPUR,
#            AALO_NARAYANGANJ, AALO_ERSHADNAGAR, AMADER_SUSASTHO

# a doctor (optional - CMED names the doctor with every consultation;
# this only puts a readable name against the id in reports)
curl -s -X POST $H/admin/doctor -H "X-Admin-Key: $A" \
  -H "Content-Type: application/json" \
  -d '{ "doctor_id": "DR0042", "hospital_id": "AALO_DHOLPUR",
        "full_name": "Dr Rahman", "active": true }'

# CMED's Channel B key - PRINTED ONCE, NEVER RECOVERABLE
curl -s -X POST $H/admin/cmed-key -H "X-Admin-Key: $A" \
  -H "Content-Type: application/json" \
  -d '{ "label": "cmed-production", "created_by": "AIMS LAB" }'
```

You should see: each clinic: `{"status":"ok","hospital_id":"AALO_DHOLPUR","cmed_hospital_id":"..."}`. The key call: `{"cmed_key":"...","label":"cmed-production",...}`.

If you see this instead	What it means, and what to do
`401 invalid admin key`	The A= line was run in another folder. `cd` into `deploy/uiu` and run it again.
`422` **with** `field required`	The JSON body is missing a field, or a quote was lost in copying. Retype the body.
`400 timezone ... is not a known zone name`	Use `Asia/Dhaka` exactly.

Capture the CMED key the moment it prints and send it to CMED over a channel that is not plain email if you can. It cannot be shown again. To replace it: issue a new one with a new label, let CMED switch, then revoke the old label with `POST $H/admin/cmed-key/revoke` and body `{"label": "cmed-production"}`.

`cmed_hospital_id` **is the blocking prerequisite**
Without it, a visit CMED describes cannot be tied to the clinic it happened in: the record is stored, **CMED receives a normal 202**, an `unmapped_clinic` alert is raised, and the recording is kept but never gets its clinical record. **Get CMED's seven codes before this step.** If they are late, register the clinics anyway and repeat the same call with the code later; it updates, it does not duplicate.

Step 11 · Install the clinic PCs against the new address

A recorder is enrolled to **one clinic and one server**. Moving a PC to this server means a fresh single-use token and one run of the installer. **Do one PC first, record a full test consultation, and only then do the rest.**

11a · Mint the tokens (on the droplet)

The fleet is described in a CSV, `backend/scripts/laptops.csv`: **one row per laptop, not per doctor**. **Add rows; never delete rows** — a trimmed file reads as clinics that no longer exist.

```
#   hospital_id,hospital_name,room,doctor_id,doctor_name
#   AALO_DHOLPUR,"Aalo Clinic, Dholpur",Room 3,,
#   AALO_DHOLPUR,"Aalo Clinic, Dholpur",Room 4,,
nano ~/AIMScribe_v3/backend/scripts/laptops.csv      # Ctrl+O, Enter, Ctrl+X

cd ~/AIMScribe_v3/backend
AIMS_BACKEND_URL=https://aimscribe.uiu.ac.bd \
AIMS_ADMIN_KEY=$(grep ^AIMS_ADMIN_KEY= ../deploy/uiu/.env | cut -d= -f2) \
  python3 scripts/mint_enrolment_tokens.py scripts/laptops.csv
```

You should see: `backend https://aimscribe.uiu.ac.bd`, one `hospital ... ready` per clinic, and a folder `scripts/enrolment_<date>_<time>/` with one instruction sheet per PC. Unused tokens expire after 14 days.

If you see this instead	What it means, and what to do
`AIMS_BACKEND_URL is not set`	Run it again exactly as shown, with the `AIMS_BACKEND_URL=` line.
`AIMS_ADMIN_KEY is not set`	The path to `.env` was wrong. Run it from `~/AIMScribe_v3/backend`.

Copy the instruction sheets to your laptop with `scp -r`, then **delete the folder from the droplet**: the tokens are credentials until they are used.

11b · Prove the keys belong together, then build the installer

REQUIRED before building the installer and before enrolling any PC
On 15 October 2026 the public keys in the repository's `recorder/keys` did not belong to the development server's private keys. An installer built from them makes **every recording fail its grant check**, and no PC ever frees its disk. Three fingerprints must be the same: the server's, the installer's, and the first PC's. No private key leaves the droplet to check this.

1. On the droplet: the server's private keys against the public halves made in 6d.

```
cd ~/AIMScribe_v3/deploy/uiu
docker compose run --rm --no-deps --user root \
  -v ~/keys:/keys:ro -v ~/AIMScribe_v3/tools:/tools:ro \
  api python /tools/check_key_pair.py --from-env --keys /keys
```

You should see: `grant MATCH <16 characters>` and the same for `receipt`. Write the two fingerprints down.

2. On your laptop, with the two files from 6d in `recorder\keys`:

```
PS> cd <your AIMScribe_v3 folder>
PS> python tools\check_key_pair.py --show --keys recorder\keys
```

You should see: the same two fingerprints as on the droplet. **If either differs, stop**: copy the files from 6d again. Then build `AIMScribeSetup.exe` as usual; the installer copies these keys to every PC it runs on.

3. On the first clinic PC, after 11c: `python tools\check_key_pair.py --show --keys C:\ProgramData\AIMScribe\keys` (from a copy of the tools folder) shows the same two fingerprints.

11c · On each clinic PC

	Do this	Note
1	**Only for a PC already enrolled to another AIMS LAB server:** wait until it has nothing left to upload (the folder `C:\ProgramData\AIMScribe\spool` has no session folders). Then delete `C:\ProgramData\AIMScribe\state\device.json`	**Without this the PC stays enrolled to the old server** and ignores the new token. A brand-new PC skips this row
2	Run `AIMScribeSetup.exe` as administrator	
3	**Backend URL:** `https://aimscribe.uiu.ac.bd`	exactly, with https
4	**CMED web address:** the address the CMED dashboard is opened from	the page's origin
5	**Enrolment token:** that PC's token from its sheet	one PC, one token
6	Finish. The agent starts and enrols itself on first start	needs the internet once

You should see: on the droplet, `docker compose logs api | grep -i enrol` shows the PC enrolling, and from then on its heartbeat every 30 seconds.

If you see this instead	What it means, and what to do
The installer says "leave empty — already enrolled"	Row 1 was skipped. Cancel, delete `device.json`, run it again.
The PC refuses to record: "not registered with AIMS LAB"	Enrolment failed. Usually an expired or already-used token: mint a new one for that row.
Recording starts but is never authorised	The PC's public keys do not match `.env`. Run the three checks in 11b, rebuild the installer with the files from 6d, and run it again on that PC.

Tell CMED the new host for their server's setting (Part A, section 3). That is the only change on their side.

Step 12 · Backups, the restore drill, and email alerts

The stack's `backup` service makes an encrypted dump of both databases every night at `AIMS_BACKUP_AT` (02:30 Dhaka time) into `/srv/aimscribe/backup`, and keeps 30 days. It runs in its own small image, `deploy/uiu/backup/Dockerfile`: the plain `postgres:16-alpine` it used before could neither read its schedule nor encrypt, and never made a backup. Make one now, to prove it, rather than waiting for tonight.

12a · The first backup, now

```
cd ~/AIMScribe_v3/deploy/uiu
docker compose run --rm backup --now
ls -lh /srv/aimscribe/backup
docker compose logs --tail 3 backup        # the nightly service's own plan
```

You should see: `aims_recordings -> aims_recordings_<date>_<time>.sql.gz.enc` and the same for `aims_clinical`, then `done; 2 backup file(s) held`. The service's log ends `next backup in <h>h <m>m` — a real wait, never a negative one.

If you see this instead	What it means, and what to do
`service "postgres" is not running`	Start the stack first (step 8).
`Permission denied` **writing** `/backups`	Step 7's last `chown` was skipped.
`is only ... bytes; keeping it for inspection`	The dump failed or the database is empty. Read the first error above it.
`AIMS_BACKUP_KEY is set but this image has no openssl`	The service is still on the plain image. `docker compose up -d --build backup`.

12b · The restore drill

Restore the newest backup into a **scratch** database, count what is in it, and drop it. The script refuses to write into either live database.

```
cd ~/AIMScribe_v3/deploy/uiu
F=$(ls -t /srv/aimscribe/backup/aims_recordings_*.enc | head -1); echo $F
docker compose run --rm backup restore /backups/$(basename $F) aims_restore_test

docker compose exec -T postgres psql -U aims_admin -d postgres \
  -c "DROP DATABASE aims_restore_test"
```

You should see: `Decompressed ...: <n> lines`, `CREATE DATABASE`, then a table of row counts with `sessions` matching the dashboard, then `DROP DATABASE`.

If you see this instead	What it means, and what to do
`bad decrypt`	The key in `.env` is not the one the file was made with. This is exactly why the key is stored off the machine.
`Restore failed; last lines:`	Read them. Do not ignore it: fix it today.
`Refusing to restore over the live database`	As intended. Restore into a new name, check it, then swap it in by hand.

A backup nobody has restored is not a backup. Do this drill today, then once a month. The backups live on the droplet's own disk, which the weekly DigitalOcean backup also copies; for a copy off the droplet, download the newest `.enc` files with `scp` once a week (Part E).

12c · Email alerts from DigitalOcean

	Do this in DigitalOcean	Emails the owner when
1	Monitoring → **Create alert policy**: Disk utilisation above **80%** for 5 min, droplet `aimscribe-prod-blr1`	the disk is filling
2	Same: Memory above **90%** for 10 min	memory is short
3	Same: CPU above **90%** for 15 min	the machine is overloaded
4	Monitoring → **Uptime** → New check: `https://aimscribe.uiu.ac.bd/health`, from the region nearest Dhaka	**the server stops answering**, and when the certificate nears expiry

PART D

Proving every endpoint

The checks to run after Part C, and to send CMED as evidence

8 · The twelve checks

	Check	Command or action	Pass
1	The name resolves	`nslookup <host> 1.1.1.1`	the droplet's IP
2	The certificate is real	`curl.exe -sI https://<host>/health`	`HTTP/1.1 200`, no TLS error
3	**Health, from outside**	`curl.exe -s https://<host>/health`	`"status":"healthy"`
4	It is the right system	the same reply	`system` **starts with** `AIMScribe v3`
5	The key is enforced	POST API 2 with no key	**401**
6	**API 2 accepts**	POST API 2 with the key	**202** `ACCEPTED`
7	**A repeat is recognised**	POST the identical body again	**200** `ALREADY_RECEIVED`
8	**API 3 Channel B accepts**	POST the prescription	**202** `ACCEPTED`
9	Only 80 and 443 are open	from your laptop: `Test-NetConnection <host> -Port 5432`	`TcpTestSucceeded : False`
10	The visits list answers, and only with the key	`curl.exe -s https://<host>/api/v2/clinical/patient/TESTP0001/visits`, then with `-H "X-CMED-Key: ..."`	**401**, then **200** `"code":"OK"`
11	The recorders' new endpoints exist and are guarded	`curl.exe -s -X POST https://<host>/api/v2/device/health -H "Content-Type: application/json" -d "{}"`	**401** `device token required` (a 404 means the old code)
12	**A real consultation matches**	one PC, CMED sending all four messages	**confirmed** on the dashboard

Checks 3 to 8 in one command, from a laptop with the repository and Python:

```
PS> python tools\channel_b_test.py --server https://aimscribe.uiu.ac.bd --key <CMED key>
```

Keep the output and send it to CMED: it is the evidence that our side is ready. Its records use test identifiers (`TESTP0001`, `CMED-TEST-01`); they confirm nothing and raise one harmless `unmapped_clinic` alert.

And then the one check that actually matters

Checks 1 to 11 prove the gateway works. **They do not prove the integration works.** Record one real consultation on one clinic PC with CMED sending all four messages, and **confirm that the recording was matched and archived**. Every other check can pass while the five identifying fields silently fail to match — and then the recording is kept with no clinical record attached.

9 · Where things are on the droplet

What	Path	Note
The code and the stack	`/home/aims/AIMScribe_v3/deploy/uiu`	where every `docker compose` command is run
The settings and the keys	`…/deploy/uiu/.env`	**mode 600. Never commit it**
The 16 GB database profile	`…/deploy/uiu/postgres/postgresql.16gb.conf`	part of the code; chosen in 6e
The databases	`/srv/aimscribe/db`	grows 20–40 GB a year
The archive	`/srv/aimscribe/archive`	**clinic / doctor / date / consultation**. Each WAV leaves this disk 2 days after its cloud copy is proven (6e); its JSON and manifest stay
Nightly backups	`/srv/aimscribe/backup`	encrypted, 30 days
The permanent cloud copy	R2 bucket `aimscribe-archive`	encrypted with `AIMS_COPY_KEY`

PART E

Daily and weekly checks

Five minutes a day keeps a silent failure from becoming a lost month

10 · Every clinic day (5 minutes)

```
ssh aims@<droplet-ip>
cd ~/AIMScribe_v3/deploy/uiu
A=$(grep ^AIMS_ADMIN_KEY= .env | cut -d= -f2); H=https://aimscribe.uiu.ac.bd/api/v2

docker compose ps                    # nine Up, four (healthy)
df -h /srv | tail -1                       # Use% below 80
curl -s $H/dashboard/summary -H "X-Admin-Key: $A" | jq .
curl -s "$H/admin/alerts?limit=20" -H "X-Admin-Key: $A" | jq .
```

Look at	Healthy	Act when
Services	nine Up	**anything restarting or exited**
Disk	under 80%	**over 80%**: archived WAVs leave 2 days after their copy is proven, so a rising disk means copies are not being proven — look for `cloud_copy_overdue` in the alerts
Unconfirmed recordings	zero once any outage is over	**any left**: kept and archived, but CMED's record never arrived or did not match. Ask CMED to check its outbox for that visit. Nothing is ever deleted for this
Offline periods	dashboard → Recorders → *Offline this week*	a room offline for hours, or *microphone silent at times*: look at that PC's network or microphone
Silent recorders	every clinic PC reported recently	a PC silent during clinic hours: killed agent or no network
Alerts	nothing new	`unmapped_clinic`, `chain_entry_rejected`, `spool_critical`, `authorisation_held`, `start_abandoned`, `reassign_not_applied`: investigate the same day. `possibly_two_consultations`: check the recording and split it if two patients ran together

11 · Every week (20 minutes)

	Check	How
1	Last night's backup exists	`ls -lt /srv/aimscribe/backup | head -3`
2	A copy of the backups off the droplet	PS> `scp aims@<droplet-ip>:/srv/aimscribe/backup/*.enc .` to a UIU machine; delete copies older than 30 days
3	Cloud copies are arriving	Cloudflare → R2 → `aimscribe-archive`: object count grew
4	Security updates applied	`cat /var/run/reboot-required` — if it exists, reboot **after clinic hours**
5	The certificate	Uptime check green; nothing from Let's Encrypt in the owner mailbox
6	Costs	DigitalOcean and Cloudflare billing pages, against the plan

12 · Every month

The restore drill (step 12b). Then write the date and the row counts in the access sheet. If it fails, it is the most important problem on the list until it is fixed.

13 · The dashboard page

The dashboard is `https://<host>/api/v2/dashboard/` and asks for the admin key, which it keeps in memory for that tab only. The gateway gives this one path a policy that allows the page's own script and style files and its own API calls, and nothing else; every other path stays closed to browsers. The JSON calls in section 10 carry the same numbers.

PART F

Updates, rollback, what was fixed, go-live

What to do later, and what to settle before the clinics depend on it

14 · Updating to a new release

```
cd ~/AIMScribe_v3/deploy/uiu
git log -1 --oneline > ~/before-update.txt     # where you were, for rollback
git pull
ls ../../backend/scripts/                      # any NEW numbered .sql file?
docker compose up -d --build
docker compose ps
```

Outside clinic hours. A consultation in progress survives an API restart — the gateway holds requests for up to 30 seconds and the recorder keeps its pieces and retries — but there is no reason to test that.

A new migration is not applied by itself. The schema scripts run only on the very first start of an empty database; a new droplet built with this manual already has every one. **A database created before this release needs three applied by hand**, in order, as the role that owns the tables — `013_v3_reassign.sql` (recordings from older recorders that move to the patient their prescription named), `014_v3_device_health.sql` (the recorders' offline log) and `015_v3_consultation_parts.sql` (which patient was opened at which second of a recording, and the per-patient files cut from it). Without 015, every recording that has a second patient click or a prescription signal stops at that step and never completes; without 014, the offline log is refused and stays on the laptops.

```
U=$(docker compose exec -T postgres printenv POSTGRES_USER)
for f in 013_v3_reassign.sql 014_v3_device_health.sql \
         015_v3_consultation_parts.sql; do
  docker compose exec -T postgres psql -v ON_ERROR_STOP=1 -U "$U" \
    -d aims_recordings -c "SET ROLE aims_recordings" -f /aims/scripts/$f
done
# check: one line must come back
docker compose exec -T postgres psql -U "$U" -d aims_recordings -tAc \
  "select to_regclass('device_health_log'), to_regclass('consultation_parts'), \
   pg_get_constraintdef(oid) \
   from pg_constraint where conname = 'chain_entries_entry_type_check'"
```

You should see: `device_health_log`, `consultation_parts` and a CHECK that lists `'reassign'` and `'marker'`. Run the scripts again and nothing changes: all three are safe to repeat.

Order: the server first, then the recorders. A new recorder sends each patient click and prescription signal to `/session/marker`, and its offline log; an old server answers 404, and a new one without 015 answers 500. The recorder keeps them waiting until the server is ready — nothing is lost — but nothing after that step in the recording uploads until then. So update the server, apply 013, 014 and 015, then roll the installer out to the clinic PCs.

15 · If it has to be undone

	Situation	What to do
1	A release misbehaves	`git checkout <the commit in ~/before-update.txt>` then `docker compose up -d --build`. Return to `main` with `git checkout main` when fixed
2	The deployment fails before any clinic uses it	**Nothing to undo.** Destroy the droplet. The old stack is untouched
3	It works, but a problem appears on day one	Re-run the installer on the PCs with the old address and fresh tokens from the old server, and tell CMED to revert their setting. **Keep the droplet running** — it holds recordings the old one does not
4	The droplet is lost entirely	Restore DigitalOcean's weekly backup as a new droplet, or rebuild with this manual and restore last night's database backup. **The cloud copies in R2 are unaffected** — if `AIMS_COPY_KEY` was kept off the machine

16 · What was found, and what is fixed

Found while checking this manual against the code on 15 October 2026, and fixed in the code the same week. Listed so that anyone who read an earlier copy knows the workaround is gone.

	Was	Now
1	Every archived WAV stayed on the server disk: full in about 9 days	Removed from this disk `AIMS_ARCHIVE_LOCAL_RETENTION_DAYS` days (2 here) after its cloud copy is proven, and never without one. JSON, manifest and catalogue stay; the manifest says where the copy is
2	The backup service never made a backup, and could not encrypt	Its own image with `openssl`; the schedule is plain arithmetic. Step 12, run with `--now` to prove it
3	The cloud copy was FLAC	**The raw WAV**, as agreed, encrypted, and decrypted again and hashed before upload. Older FLAC copies still restore
4	The dashboard loaded blank through the gateway	A policy for that path only; checked in a browser behind Caddy
5	`/health` said storage was connected without asking	It asks both buckets, every 30 seconds at most, and says which one failed
6	The monitor's 500 GB floor alarmed for ever on a 320 GB disk; two floors never reached it	Each floor capped at 15% of its disk; all three passed through
7	The gateway did not start: its log filter named a module Caddy does not have	Fixed, and the whole Caddyfile validated in `caddy:2.8-alpine`
8	The repository's public keys did not match the server	11b now proves the pair before the installer is built
9	A recording without CMED's API 2 was deleted after 24 hours	**Kept and archived**; CMED's record attaches whenever it arrives. Only a patient's refusal erases
10	A server "no" — a wrong key, an unmapped clinic — deleted the recording on the laptop	**Held on the laptop** and asked again every 5 minutes; the fix releases it
11	A wrong patient opened first locked the room until Stop	The prescription decides: the whole recording moves to that patient (013)
12	An offline laptop's heartbeats and alerts were lost	Logged on the laptop, delivered on reconnect, then deleted there (014)

17 · Go-live checklist

	Item	Where
■	Both accounts owned by your-team-email@example.org, two-factor on, recovery codes stored	Part 0
■	Billing alert set on DigitalOcean	0.2
■	Two R2 buckets, two tokens, public access off	6
■	Droplet in BLR1, `s-8vcpu-16gb`, backups and monitoring on	step 1
■	Name resolves to the droplet	step 2
■	Firewall: 22 from your IP only; 80 and 443	step 3
■	Root and password logins off; timezone Asia/Dhaka; swap on	step 4
■	Every `ok` in 6g; `compose settings: OK`	step 6
■	**Six secrets stored off the machine**	6g
■	**Key fingerprints identical** on the droplet, the laptop and the first PC; installer rebuilt	6d, 11b
■	Archive folder owned by 10001	step 7
■	Nine services up; certificate obtained	step 8
■	`/health` healthy **from outside**	step 9
■	**All seven clinics registered with CMED's codes**	step 10
■	CMED key issued and delivered; CMED has the host	step 10, Part A
■	One PC installed and a real consultation **confirmed**	step 11, Part D
■	First backup made with `--now`, and **restored**	step 12
■	Email alerts and the uptime check active; webhook set	12c, 6f
■	`/health` shows both buckets `connected`	step 9
■	Schema scripts 013 and 014 present (automatic on a new database; by hand on an old one)	14
■	Checks 10 and 11 answer 401, not 404	Part D
■	channel_b_test output sent to CMED	Part D

18 · One page to remember

Question	Answer
Which compose file?	`deploy/uiu/docker-compose.yml` — the server stack, despite the folder name
Which droplet?	`s-8vcpu-16gb`**, Bangalore**, Ubuntu 24.04, backups and monitoring on
What must exist first?	**A domain name pointing at the droplet**
Which ports open?	**80 and 443 only**, plus SSH from your own IP
How many buckets?	**Two**: `aimscribe-segments` and `aimscribe-archive`
What must change for 16 GB?	`AIMS_PG_CONF`, `AIMS_PG_MEMORY`, `AIMS_ARCHIVE_MEMORY`, `AIMS_WORK_TMPFS`
Where do CMED's clinic codes go?	`cmed_hospital_id` on each clinic. Step 10
Which secrets must leave the machine?	**Six**, led by `AIMS_COPY_KEY` and `AIMS_BACKUP_KEY`
What does CMED change?	**One setting** — our host. Their page changes nothing
What does each clinic PC need?	**One installer run** with the new address and its own token
First thing after it is up?	`curl https://<host>/health` **from somewhere else**
Most valuable half hour?	**The restore drill.** Step 12b
Who gets the notices?	your-team-email@example.org

Prepared by AIMS LAB, United International University. Companions: CMED_1 Software Integration, CMED_2 Server Deployment, and the Cloud Hosting Plan. Every port, path, service, setting and command here was checked against the stack itself.